

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIDAD CULHUACÁN

Reporte de proyecto

Ecosistema multiagente para la generación y validación de circuitos analógicos CA/CD a partir de lenguaje natural

PRESENTAN:

ANTONIO DE JESUS TORRES MARTÍNEZ

DIEGO JESUS GUTIERREZ DELGADO

ASESOR:

M. en C. RICARDO ISRAEL CALZADA SALAS



Ciudad de México 3 de noviembre de 2026

Índice

Introducción **1**

Planteamiento del Problema . . . 1
Objetivo General . . . 2
Objetivos Específicos . . . 2
Estado del Arte . . . 3
AnalogCoder . . . 3
AnalogAgent . . . 3
CircuitLM . . . 4
Justificación . . . 4
Límites y Alcances . . . 5

1. Marco Teórico **6**

1.1. Modelos de lenguaje grande . . . 6
1.2. Sistemas multiagente y orquestación . . . 7
1.2.1. LangGraph . . . 7
1.2.2. LangFuse . . . 8
1.3. Entrenamiento de Agentes para el diseño de circuitos analógicos . . . 9
1.3.1. Reglas de Prompts . . . 9
1.3.2. Generacion Aumentada por Recuperación (RAG) . . . 9
1.3.3. Spice (Simulation Program with Integrated Circuit Emphasis) . . . 11
1.3.4. PySpice . . . 11

2. Diseño y desarrollo del proyecto **12**

2.1. Diseño Conceptual . . . 12
2.2. Descomposición funcional y casos de uso . . . 13
2.2.1. Vista funcional . . . 13
2.2.2. Casos de uso . . . 14
2.3. Arquitectura del sistema (modelo C4) . . . 14
2.3.1. Nivel 1 — Contexto . . . 14
2.3.2. Nivel 2 — Contenedores . . . 15
2.3.3. Nivel 3 — Componentes del servicio de agentes . . . 16
2.4. Comportamiento dinámico: flujo de generación y validación . . . 18

2.4.1. Máquina de estados y secuencia de comunicación 18
2.4.2. Lazo de validación del agente Curador 19
2.5. Desarrollo del sistema 21
2.5.1. Modelo entidad-relación de la base de datos 21
2.5.2. Catálogo de circuitos, ajuste a valores definidos y modo de análisis . 21
2.6. Pantallas del prototipo funcional 23
2.6.1. Pantalla de inicio de sesión y ruta de solución 23
2.6.2. Configuración de proveedores de IA y modelos por agente 23
2.6.3. Espacio de trabajo, nueva solicitud y avance en vivo 24
2.6.4. Archivos generados y Visualizador de circuitos 24

3. Pruebas y Análisis de Resultados **26**

4. Conclusiones y Trabajo Futuro **27**

Referencias **28**

A. Diagramas de casos de uso **30**

Índice de figuras

2.1. Diseño conceptual del sistema en cuatro fases: describir, diseñar, verificar y entregar. 13
2.2. Diagrama funcional del sistema. 14
2.3. C4 Nivel 1: Diagrama de contexto del sistema. 15
2.4. C4 Nivel 2: Diagrama de contenedores del sistema. 16
2.5. Subgrafo de Cálculo: despacho Maestro-Trabajador en paralelo y ciclo de vida de cada trabajador. 17
2.6. C4 Nivel 3: Componentes del servicio de agentes. 17
2.7. Máquina de estados finita y función de transición para el control de flujo entre agentes. 19
2.8. Diagrama de secuencia del flujo principal de generación y validación. 19
2.9. Flujo de evaluación, decisión y ajuste del agente Curador. 20
2.10. Diagrama Entidad-Relación (ERD) de la base de datos en PostgreSQL. 21
2.11. Pantalla de inicio de sesión con el diagrama de la ruta de solución. 23
2.12. Pantalla de configuración de proveedores de IA y asignación de modelos por agente. 23
2.13. Espacio de trabajo principal con proyectos, conversaciones y actividad reciente. 24
2.14. Pantalla de creación de nueva solicitud en lenguaje natural. 24
2.15. Panel de seguimiento en vivo de las etapas de ejecución y de los ajustes del Curador. 24
2.16. Vista del repositorio de archivos y circuitos SPICE generados. 24
2.17. Visualizador de circuitos con diagrama esquemático, código SPICE, lista de materiales, gráfica de simulación y explicación del diseño. 25

Introducción

Planteamiento del Problema

Con el avance tecnológico, se le han encontrado muchas utilidades a los Modelos de Lenguaje de Gran tamaño. En este sentido la generación de circuitos analógicos mediante lenguaje natural tiene un percance, por la manera en que funcionan los LLMs, rara vez devuelven una solución satisfactoria puesto que trabajan mediante distribuciones probabilísticas y no por leyes físicas. Esta manera de funcionar resulta en defectos en la generación del circuito, sumando a que utilizar un mismo agente/LLM tiene una deficiencia en la memoria, creando ambigüedad y alucinaciones del mismo.

A diferencia del diseño digital, que se apoya en abstracciones estandarizadas y flujos de automatización maduros, el diseño de circuitos analógicos descansa en el conocimiento experto, el razonamiento heurístico y un ajuste iterativo guiado por simulación [1]. Ese ciclo —proponer un circuito, simularlo, revisar los resultados y volver a ajustar— se repite numerosas veces antes de obtener un diseño que cumpla las metas, y validar un diseño completo puede exigir largos periodos de iteración [2]. Buena parte de ese conocimiento reside en la experiencia de quien diseña y no en un procedimiento que pueda seguirse paso a paso, lo que vuelve el diseño analógico difícil tanto de automatizar como de aprender.

Los Modelos de Lenguaje Grande (LLM) abrieron la posibilidad de describir un circuito en lenguaje natural y recibir una propuesta de diseño de forma automática. Al llevarla a la práctica, sin embargo, aparecen limitaciones documentadas. Estos modelos alucinan componentes, violan restricciones físicas del circuito y producen salidas eléctricamente inconsistentes [3], por lo que sus diseños rara vez funcionan en el primer intento, ya que el modelo no comprueba por sí mismo si lo que generó cumple.

Los trabajos publicados se reparten entre dos líneas. La primera diseña a nivel de transistor, atado a un proceso de fabricación específico descrito por su *Process Design Kit* (PDK), y emplea la simulación SPICE como realimentación durante el diseño; es la línea más desarrollada [1], [4], [5]. La segunda trabaja a nivel de placa, con componentes comerciales, y verifica el resultado mediante reglas eléctricas [2], [3], en parte porque no todo componente comercial cuenta con un modelo SPICE utilizable, lo que impide simular cualquier diseño de forma uniforme. La simulación SPICE a nivel de placa ha comenzado a aparecer, pero en forma de banco de prueba que compara el diseño generado contra un circuito de referencia construido por expertos [6], y no como una señal que el propio sistema utilice para ajustar el

diseño durante su ejecución. Entre los trabajos revisados no se encontró un sistema generativo de varios agentes que use la simulación SPICE como realimentación dentro del lazo de diseño, con un curador que decida el ajuste, orientado a circuitos analógicos CA/CD con componentes comerciales. Ese es el punto del espacio de diseño que este proyecto se propone ocupar.

Objetivo General

Desarrollar y evaluar un ecosistema multiagente basado en LLMs y Generación Aumentada por Recuperación (RAG) para la síntesis, dimensionamiento y verificación automatizada de circuitos analógicos CA/CD con componentes comerciales (COTS) a partir de lenguaje natural, garantizando su convergencia eléctrica funcional mediante simulación en lazo cerrado con ngspice.

Objetivos Específicos

1. **Desarrollar** una interfaz web modular con autenticación y gestión de credenciales basada en Better Auth y Drizzle ORM sobre PostgreSQL/Neon, que permita al usuario parametrizar restricciones de diseño analógico en lenguaje técnico y visualizar de forma integrada el código netlist generado, la documentación funcional del circuito, el esquema resultante y los gráficos derivados de la emulación eléctrica.
2. **Entrenar e implementar** un ecosistema multiagente coordinado mediante grafos de estado en LangGraph, integrando un módulo de cálculo y un agente curador sustentado en Generación Aumentada por Recuperación (RAG) para consultar especificaciones de componentes comerciales (COTS), gobernado por un lazo de retroalimentación acotado a un presupuesto estricto de máximo 10 iteraciones de auto-corrección.
3. **Desplegar** una arquitectura de microservicios desacoplada en contenedores Docker independientes para el cliente y el servidor stateless, configurando la pasarela de llamadas hacia APIs externas de LLMs y habilitando la ejecución segura del motor ngspice mediante PySpice en un entorno de pruebas aislado (sandbox).
4. **Evaluar** el desempeño global del sistema mediante un banco de pruebas de circuitos CA/CD a nivel COTS, cuantificando la tasa de convergencia en simulación (solve rate dentro de las 10 iteraciones permitidas) y la precisión de las restricciones eléctricas frente a las metas solicitadas.

Estado del Arte

La automatización y el diseño generativo de circuitos analógicos mediante inteligencia artificial han cobrado especial relevancia en años recientes. Para situar el aporte de este trabajo, resulta indispensable analizar las metodologías propuestas en la literatura técnica reciente.

AnalogCoder

En primer lugar, **AnalogCoder** [4] introdujo la propuesta de diseñar circuitos tratándolos como código ejecutable; en lugar de pedirle al modelo de lenguaje que describa un circuito en lenguaje natural, se le instruye para que escriba un script en Python capaz de generarlo. Al convertir el diseño en un programa ejecutable se reduce significativamente la ambigüedad de la síntesis y el netlist resultante puede simularse de inmediato. El sistema funciona sin necesidad de reentrenar el modelo base y comprueba recursivamente cada propuesta con una secuencia de verificaciones por simulación SPICE, validando que el circuito simule sin nodos flotantes, que los puntos de operación en corriente continua sean correctos y que la respuesta dinámica o de frecuencia cumpla con las especificaciones. Cuando una verificación falla, la traza de error regresa al modelo para orientar el siguiente intento de corrección. No obstante, esta aproximación coordina todo el flujo con un solo agente y se limita a nivel de transistor sobre un proceso de fabricación específico.

AnalogAgent

Por su parte, **AnalogAgent** [1] reparte esta carga de trabajo mediante un esquema de colaboración multiagente. Su contribución central consiste en la gestión y destilación del conocimiento acumulado durante los fallos: cuando un intento de diseño resulta fallido, el sistema procesa el error para traducirlo en una regla heurística explícita que se incorpora a una memoria auto-evolutiva y se recupera en iteraciones futuras, reduciendo el número total de llamadas al modelo. Adicionalmente, el estudio documenta una limitación típica de los agentes monolíticos basados en modelos de lenguaje: a medida que el historial de chat crece con cada intento de corrección, el resumen de contexto tiende a diluir u omitir detalles e instrucciones técnicas críticas. Esta estrategia de acumular conocimiento para guiar los pasos subsiguientes influye directamente en la concepción del agente curador de este trabajo, aunque mantiene la limitación de operar a nivel de transistor.

CircuitLM

Finalmente, **CircuitLM** [3] comparte con este proyecto el nivel de abstracción a nivel de componentes y módulos comerciales a partir de especificaciones de lenguaje natural. Este sistema emplea una tubería multiagente dedicada a la identificación de componentes, la consulta de hojas de especificaciones y la estructuración del esquema. No obstante, se orienta a electrónica digital y embebida —microcontroladores como Arduino o ESP32— y difiere del presente trabajo en su método de validación: en lugar de integrar simulación SPICE en el lazo, verifica la topología con un motor determinista de reglas eléctricas y un modelo de lenguaje supervisor. Sus propios autores señalan la verificación con simulador en el lazo como una limitación pendiente, justamente la señal de realimentación que este proyecto incorpora.

Ubicación de este Proyecto

Junto a los desarrollos mencionados, otras propuestas complementan el panorama del área. **AnalogSAGE** [5] introduce memoria estratificada y mecanismos de autoevolución a la síntesis a nivel de transistor; **PCBSchemaGen** [2] extiende la verificación estática de reglas al ámbito de tarjetas de circuito impreso reales; y **HWE-Bench** [6] provee un entorno de evaluación comparativa que mide el rendimiento de netlists generados contrastándolos con diseños de referencia de expertos a través de simulaciones SPICE. Adicionalmente, se registran contribuciones como **ADO-LLM** [7] para la optimización bayesiana en el diseño físico y **SPICEPilot** [8] para la generación asistida de scripts de control.

Al contrastar las soluciones se observa que ninguno de los antecedentes recopilados integra simultáneamente los cuatro rasgos propuestos en este ecosistema. Mientras que los sistemas orientados a transistores explotan la simulación en lazo cerrado y el ajuste paramétrico iterativo, no contemplan el uso de componentes comerciales. Por el contrario, los enfoques comerciales evitan la simulación dinámica en el lazo de optimización o la reducen a una métrica de prueba externa. Este proyecto une la retroalimentación física por simulación de los flujos a nivel de transistor con el diseño a nivel de placa mediante componentes comerciales, implementando una arquitectura multiagente coordinada por un curador entrenado por aprendizaje por refuerzo.

Justificación

Planteamos un método diferente de utilizar múltiples agentes en un solo módulo, aplicando procesamiento en paralelo, capaces de sintetizar, dimensionar y verificar circuitos

electrónicos CA/CD de manera automatizada mediante lenguaje natural. Todo mediante una implementación en la nube, descartando la necesidad de un software propietario especializado.

La arquitectura propuesta permite que el sistema sea utilizado para un publico abierto, sin embargo, por la naturaleza del proyecto permite a estudiantes relacionados con circuitos electrónicos analógicos CA/CD tener un aditamento en sus estudios y preparación académica.

Límites y Alcances

El sistema abarca el recorrido completo desde una descripción en lenguaje natural hasta un netlist simulado en ngspice. Del lado del cliente, gestiona el registro y el inicio de sesión, los espacios de trabajo y las claves con las que el usuario envía sus solicitudes. Del lado del servidor, lee la descripción y extrae de ella los parámetros eléctricos, divide los circuitos de varias etapas en sub-bloques, calcula los valores de los componentes, genera el código PySpice y el netlist, simula en ngspice, lee las métricas, ajusta los valores de forma iterativa a través del curador y entrega al usuario el netlist final junto con sus métricas y el historial de iteraciones.

El sistema trabaja únicamente con circuitos analógicos CA/CD formados por componentes comerciales que admiten un modelo SPICE, como resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales descritos por su macromodelo. Un componente sin un modelo utilizable queda fuera de su alcance, ya que sin modelo no hay simulación y, por tanto, no hay señal de realimentación. El sistema no diseña la disposición física de la placa (PCB), no realiza síntesis a nivel de transistor para un proceso de fabricación específico (PDK) ni genera esquemáticos gráficos. Por último, los valores numéricos que aparecen en los requisitos son metas de la primera versión y pueden ajustarse durante la evaluación experimental.

Capítulo 1

Marco Teórico

El sistema propuesto se sitúa en el cruce de cuatro dominios técnicos: el diseño de circuitos analógicos, los modelos de lenguaje, la Generacion Aumentada por Recuperación y la simulación física de circuitos, articulados sobre una arquitectura de software distribuida. Este capítulo expone los conceptos que sostienen cada uno de esos dominios y, sobre todo, las relaciones entre ellos, porque la aportación del proyecto no reside en ninguno por separado, sino en su integración dentro de un lazo de generación y validación. La exposición sigue el recorrido de los datos en el sistema: desde la descripción en lenguaje natural hasta el netlist validado.

1.1. Modelos de lenguaje grande

Un modelo de lenguaje grande es un modelo estadístico, entrenado sobre grandes volúmenes de texto, que estima la probabilidad del siguiente elemento de una secuencia a partir de los anteriores. La generación procede de forma autorregresiva: el modelo produce un elemento, lo incorpora al contexto y repite el proceso, de modo que la salida se construye paso a paso [12]. Esta capacidad de generación permite que el modelo transforme una descripción en lenguaje natural en una estructura de datos o en código, que es precisamente lo que el sistema necesita en dos puntos: la extracción de los parámetros eléctricos a partir de la descripción del circuito y el cálculo de los valores de los componentes de cada sub-bloque. Internamente, el modelo procesa el texto como una secuencia de unidades llamadas tokens y opera sobre una ventana de contexto de tamaño acotado, que fija cuánta información puede considerar en una sola consulta; esta limitación reaparece más adelante como una de las razones para descomponer el circuito en sub-bloques.

Dos propiedades de estos modelos resultan pertinentes para el diseño. La primera es el aprendizaje en contexto: el modelo ajusta su respuesta a partir de los ejemplos e instrucciones incluidos en la propia consulta, sin necesidad de reentrenarlo [13]. Esto permite proporcionarle las ecuaciones de diseño de un sub-bloque como contexto y obtener los valores correspondientes. La segunda es la generación de salida estructurada, en ocasiones mediante mecanismos de llamada a funciones, que obliga al modelo a responder en un formato definido —por ejemplo,

un objeto JSON con el tipo de circuito y sus parámetros— y facilita el procesamiento posterior de la respuesta.

La limitación que motiva el problema de investigación es complementaria a estas capacidades. El modelo genera texto plausible, pero no comprueba por sí mismo lo que produce [14]. En el dominio de los circuitos, esto se traduce en componentes inexistentes, restricciones físicas violadas y salidas eléctricamente inconsistentes que rara vez funcionan al primer intento [1], [4]. El modelo no dispone de un mecanismo interno que contraste su propuesta con el comportamiento físico del circuito; produce una respuesta que parece correcta según los patrones del lenguaje, no según las leyes de la electrónica. Esta brecha entre plausibilidad lingüística y corrección física es la que justifica incorporar una verificación externa al modelo, y es el punto del que parte el diseño del sistema: la simulación aporta la comprobación que el modelo no realiza.

La generación del modelo no es determinista. En cada paso, el siguiente elemento se elige de una distribución de probabilidad, y un parámetro de temperatura regula cuánta variación se admite en esa elección. Una temperatura baja produce salidas más estables y, al fijarla, hace repetible la generación, condición necesaria para comparar ejecuciones durante la evaluación. Esta misma variabilidad sustenta la estrategia de referencia frente a la que se contrasta el sistema. El enfoque *best-of-N* consiste en pedir al modelo N propuestas independientes para la misma descripción, evaluarlas y conservar la mejor [15]. Es una forma directa de aprovechar el muestreo, pero no incorpora realimentación: cada propuesta se genera sin información sobre los fallos de las anteriores. El sistema propuesto se mide contra esta línea base a igual número de llamadas al modelo, lo que permite distinguir la mejora atribuible al lazo de ajuste de la que se obtendría por el solo hecho de muestrear más.

1.2. Sistemas multiagente y orquestación

1.2.1. LangGraph

Es un marco de trabajo (framework) de ingeniería de software orientado al desarrollo de aplicaciones agenticas y flujos multiagente cíclicos, modelándolos formalmente como Grafos Dirigidos o Máquinas de Estados Finitas (FSM). En su arquitectura, los nodos representan agentes o funciones computacionales, los bordes definen las transiciones lógicas (incluyendo bifurcaciones condicionales), y el estado (State) constituye un bus de memoria estructurado e inmutable.

LangGraph coordina de manera declarativa la interacción entre los diferentes agentes del subsistema servidor. Permite ejecutar el patrón Master-Worker en paralelo para sub-bloques

funcionales y gestionar bordes condicionales que reinyectan diagnósticos y habilitan lazos de optimización iterativos en caso de fallo.

1.2.2. LangFuse

Es una plataforma de ingeniería y observabilidad de código abierto (LLM Observability and Tracing Platform) diseñada para rastrear, auditar y monitorizar el ciclo de vida de aplicaciones basadas en modelos de lenguaje. Facilita la telemetría detallada de trazas de ejecución (traces y spans), registro de latencias, consumo de tokens/costes asociados y evaluación cualitativa/cuantitativa de salidas y llamadas a herramientas (tool calling).

Proporciona la infraestructura de auditoría y trazabilidad en tiempo de ejecución. Permite inspeccionar qué instrucciones exactas recibió cada agente, monitorear el costo computacional de las llamadas concurrentes en la fase Map-Reduce y correlacionar los errores o reintentos de diseño con las métricas de respuesta de cada modelo.

En tareas complejas de ingeniería analógica —como la síntesis, dimensionamiento y verificación de circuitos electrónicos—, el uso de un único Modelo de Lenguaje Grande (Single LLM Agent) genera un cuello de botella crítico: la atrición o saturación de la ventana de contexto (context attrition). A medida que se incrementa el número de directivas, restricciones topológicas y mensajes de depuración, el modelo tiende a alucinar nodos flotantes, interconexiones erráticas o componentes comerciales inexistentes. Un Sistema Multiagente (Multi-Agent System - MAS) resuelve esta limitación mediante el principio de descomposición funcional (divide y vencerás), desacoplando las responsabilidades en agentes autónomos con prompts pequeños, especializados y estrictamente aislados.

La orquestación centralizada no se implementa como un script secuencial rígido, sino como una Máquina de Estados Finita (FSM) estructurada como un grafo cíclico dirigido mediante LangGraph: Estado Compartido (State): Constituye el bus de datos inmutable y central donde se registran la especificación formal del circuito, las variables intermedias, las listas de componentes calculadas y el historial de iteraciones. Nodos y Bordes Condicionales: Los nodos encapsulan a los agentes especializados, mientras que los bordes condicionales evalúan en tiempo de ejecución las transiciones del flujo, gestionando lazos cerrados de optimización (feedback loops) y detenciones de seguridad ante fallas críticas.

Para resolver circuitos analógicos multietapa (como fuentes reguladas CA/CD compuestas por etapas de transformación, rectificación, filtrado y regulación lineal), el sistema implementa el patrón concurrente Master-Worker estructurado en dos fases

Nodo Maestro (Master):

Recibe la descripción desglosada y el plan del circuito desde el Orquestador. Su rol consiste en particionar el problema global en especificaciones locales independientes para

cada sub-bloque funcional y asegurar que ningún sub-bloque exceda los límites de tokens de la ventana de contexto permitida.

Sub-agentes Trabajadores (Workers):

Se instancian de manera concurrente. Cada Worker resuelve de forma exclusiva el dimensionamiento y el planteamiento analítico de su etapa asignada (cálculo de impedancias, selección de valores comerciales COTS, corrientes de polarización). Al no recibir el código ni los errores de las demás etapas, se elimina el consumo excesivo de memoria operativa del modelo.

1.3. Entrenamiento de Agentes para el diseño de circuitos analógicos

1.3.1. Reglas de Prompts

1.3.2. Generacion Aumentada por Recuperación (RAG)

En esta arquitectura, la Generación Aumentada por Recuperación (RAG) sustituye los esquemas basados en ensayo y error o aprendizaje por refuerzo puro, transformando el subsistema en un entorno guiado por conocimiento empírico y documental indexado.

El RAG se estructura a través del Módulo Curador de Conocimiento y opera en tres dimensiones fundamentales:

Recuperación Previa y Anclaje Técnico (Pre-Generation Retrieval)Cuando el usuario describe un circuito en lenguaje natural, el sistema no permite que los LLMs asuman parámetros arbitrarios. El mecanismo de RAG realiza una búsqueda vectorial y semántica sobre bases de datos técnicas locales: Catálogos de Componentes COTS (Commercial Off-The-Shelf): Hojas de datos (datasheets) y tablas paramétricas de componentes reales del mercado (diodos rectificadores, capacitores de filtrado, transistores BJT/MOSFET y reguladores integrados). Plantillas de Topologías Analógicas Validadas: Estructuras de referencia netlist probadas para el tipo de circuito solicitado (filtros pasivos, fuentes lineales, configuraciones de amplificadores).Directivas de Simulación para SPICE: Parámetros de convergencia y tolerancias (.OPTIONS, condiciones iniciales .IC).El Curador extrae estos fragmentos pertinentes y los inyecta en el prompt inicial del Master y de los Workers, restringiendo el espacio de diseño a parámetros realizables en hardware físico real.

B. Mitigación de Alucinaciones Topológicas y de SizingLos modelos de propósito general tienden a inventar valores ideales (p. ej., resistores de valores no normalizados) o directivas incompatibles con los motores SPICE. La inyección dinámica de contexto mediante

RAG actúa como un filtro (grounding): obliga al agente de cálculo a elegir exclusivamente modelos que cuenten con archivos de biblioteca (.lib / .model) reconocidos por el simulador ngspice.

Lazo Cerrado de Curación y Memoria de Diseño (Knowledge Curation Loop)La aportación diferencial del RAG en este sistema radica en su funcionamiento en lazo cerrado tras la ejecución física en ngspice:

Captura del Diagnóstico: Si ngspice reporta una divergencia numérica (fallo de Newton-Raphson, timestep too small) o una discrepancia severa respecto a la especificación esperada, el Agente Shell captura el log crudo. Destilación Curada: En lugar de reinyectar el mensaje de error textual íntegro al modelo generador, el Curador procesa el fallo, identifica la causa física subyacente y genera una directiva de ajuste o regla de diseño aprendida. Consolidación en la Base Vectorial: Esta heurística verificada se almacena en el almacén de conocimiento persistente (Self-Evolving Memory / Base Vectorial). En el reintento actual (o en futuros diseños de la misma familia de circuitos), el sistema recupera dicha directiva, impidiendo que el ecosistema caiga en ciclos repetitivos de error y acelerando la convergencia a una solución funcional.

El diseño de un circuito analógico no se resuelve con un procedimiento cerrado que, a partir de una especificación, devuelva una única respuesta correcta. El proyectista parte de un conjunto de metas eléctricas —una ganancia, una frecuencia de corte, un punto de polarización— y propone una topología y unos valores de componentes que, según las ecuaciones de diseño, deberían cumplirlas. La propuesta se verifica mediante simulación o medición, se comparan los resultados con las metas y, cuando hay desviación, se ajustan los valores y se repite el proceso. Este ciclo de proponer, simular, revisar y ajustar se recorre varias veces antes de que el circuito cumpla las metas [18].

Las ecuaciones de diseño son el vínculo entre el dominio de la especificación y el de la implementación. Para un amplificador no inversor con amplificador operacional, por ejemplo, la ganancia se relaciona con el cociente de dos resistencias; para un filtro RC de primer orden, la frecuencia de corte queda determinada por el producto de una resistencia y una capacitancia. Estas relaciones permiten calcular un valor inicial, pero no garantizan que el circuito real cumpla la meta, porque el modelo ideal ignora efectos que la simulación sí captura: la carga entre etapas, las no idealidades de los componentes y la interacción entre sub-bloques. De ahí que la simulación funcione como árbitro: es el mecanismo que comprueba si los valores calculados producen el comportamiento esperado.

El proyecto se restringe a circuitos analógicos de corriente alterna y corriente directa construidos con componentes comerciales —resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales descritos por macromodelo—, en contraste

con la línea de trabajo que diseña a nivel de transistor para un proceso de fabricación específico [19]. Esta distinción no es accesoria. El diseño a nivel de componente comercial trabaja con dispositivos cuyos modelos ya existen y cuyos valores se eligen de catálogos discretos, mientras que el diseño a nivel de transistor dimensiona dispositivos sobre un nodo tecnológico. La frontera entre ambos niveles delimita el alcance del proyecto y, como se argumenta en el estado del arte, marca el espacio que la propuesta ocupa.

1.3.3. Spice (Simulation Program with Integrated Circuit Emphasis)

Desarrollado originalmente en la Universidad de California en Berkeley, SPICE es el software estándar industrial y académico para la simulación numérica y análisis del comportamiento dinámico de circuitos electrónicos analógicos. Modela circuitos a partir de un listado de interconexiones (netlist) y resuelve numéricamente sistemas de ecuaciones algebraico-diferenciales no lineales empleando técnicas como el análisis nodal modificado y métodos iterativos de Newton-Raphson.

Constituye el entorno cuantitativo de validación física real (a través de motores como ngspice). Actúa como el criterio de verdad técnica indispensable para verificar parámetros como ganancia, margen de fase o rizo, y confirmar la ausencia de anomalías topológicas (nodos flotantes, cortocircuitos o fallos de convergencia).

1.3.4. PySpice

Es una biblioteca de abstracción e interfaz para Python que encapsula la sintaxis de simulación de circuitos electrónicos y establece un puente programático con motores de simulación SPICE (particularmente ngspice). Provee una interfaz orientada a objetos para declarar componentes, topologías y directivas de simulación (.AC, .DC, .TRAN, .OP) sin requerir el ensamblado manual de texto plano.

Es la herramienta empleada por el Agente de Escritura para traducir las estructuras de datos abstractas generadas por los agentes de cálculo a código de simulación estructurado y ejecutable. Asimismo, captura vectorialmente los resultados numéricos emitidos por el simulador para que el sistema evalúe el desempeño del circuito en bucle cerrado.

Capítulo 2

Diseño y desarrollo del proyecto

2.1. Diseño Conceptual

Este capítulo presenta el diseño y el desarrollo del sistema propuesto: una plataforma web conectada a un servicio de agentes de inteligencia artificial que permite diseñar, calcular con valores definidos, simular en `ngspice`, analizar y documentar circuitos analógicos de corriente alterna y corriente directa (CA/CD) a partir de una descripción en lenguaje natural.

El funcionamiento general del sistema se divide en cuatro fases: describir, diseñar, verificar y entregar. En la primera fase (*describir*), el usuario pide su circuito con sus propias palabras desde la interfaz web. En la segunda fase (*diseñar*), los agentes de inteligencia artificial interpretan el pedido, identifican los parámetros eléctricos y calculan los componentes necesarios. En la tercera fase (*verificar*), el sistema arma el circuito con valores definidos, lo simula en `ngspice` y revisa si cumple la meta; cuando hay alguna desviación, ajusta los valores y vuelve a simular hasta corregirla. Por último, en la cuarta fase (*entregar*), el usuario recibe el circuito validado con su esquema, sus gráficas, el código SPICE, la lista de materiales y la explicación del diseño, como se muestra en la figura 2.1.

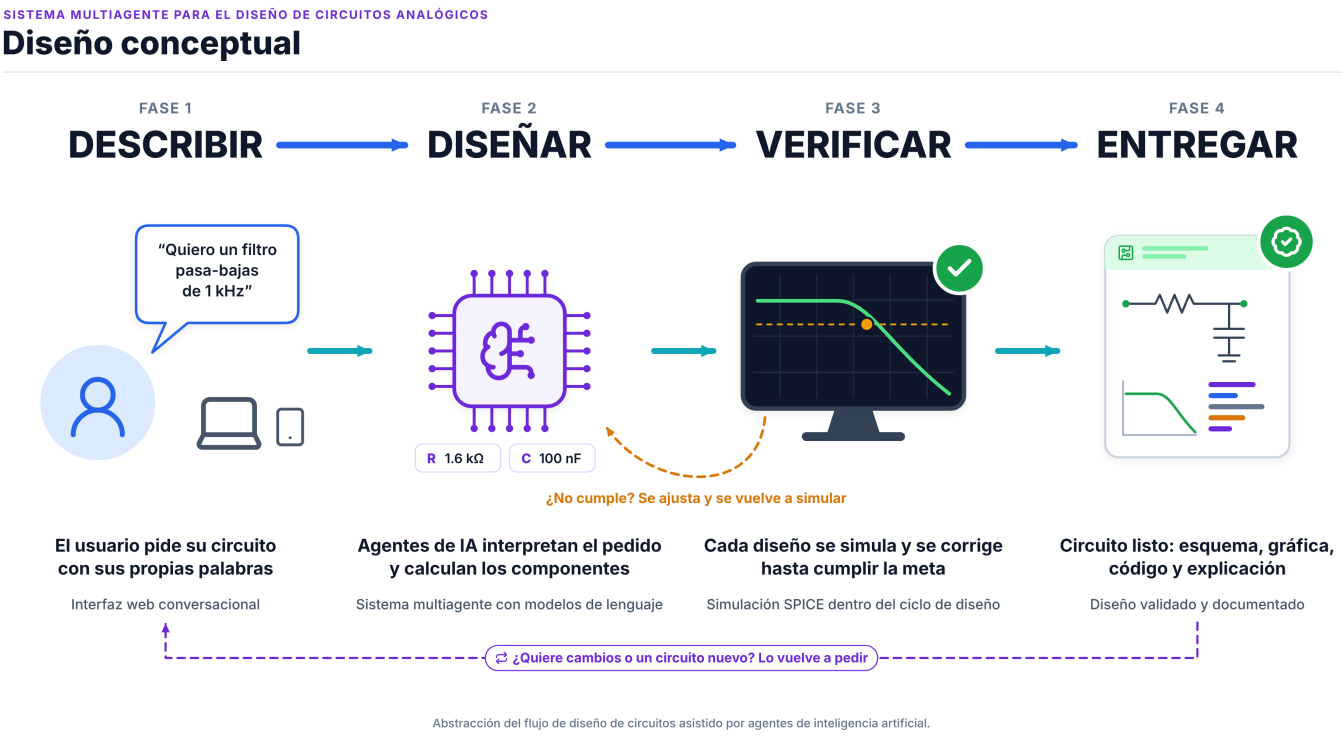


Figura 2.1. Diseño conceptual del sistema en cuatro fases: describir, diseñar, verificar y entregar.

2.2. Descomposición funcional y casos de uso

2.2.1. Vista funcional

Las tareas principales que realiza el sistema se resumen en tres requerimientos funcionales fundamentales (el listado completo se incluye en los anexos):

- **RF-01 Interpretación de la solicitud:** Recibir la descripción del usuario en lenguaje natural, identificar los parámetros eléctricos y elegir la ruta correspondiente: responder una consulta general, pedir algún dato faltante, analizar un circuito dado o diseñar un circuito nuevo.
- **RF-02 Cálculo y armado del circuito con valores definidos:** Dividir el diseño en bloques cuando consta de varias etapas, calcular en paralelo el valor de sus componentes, ajustarlos a series de valores definidos y generar el archivo SPICE listo para simular.
- **RF-03 Simulación en lazo cerrado y entrega de resultados:** Simular el circuito en `ngspice`, comparar las mediciones contra la meta solicitada para decidir si se acepta o

se ajusta el diseño, y entregar al usuario el circuito final junto con su lista de materiales, gráficas y explicación.

El diagrama funcional de la figura 2.2 muestra cómo se encadenan estas funciones desde que entra el mensaje del usuario hasta que se obtiene el circuito validado.

Diagrama por agregar (SVG): `../src/diagrams/diagrama_funcional.svg`

Flujo funcional: Entrada en lenguaje natural → Interpretación (Orquestador) → Cálculo y ajuste a valores definidos → Escritura de netlist → Simulación (`ngspice`) → Evaluación (Curador: aceptar, ajustar o rechazar) → Entrega documentada.

Figura 2.2. Diagrama funcional del sistema.

2.2.2. Casos de uso

En el sistema intervienen dos tipos de actores: el usuario (estudiante o ingeniero electrónico), que interactúa con la aplicación web, y los servicios externos (el proveedor de modelos de inteligencia artificial y Langfuse), que el sistema consulta durante cada corrida.

Desde la aplicación web, el usuario puede crear su cuenta e iniciar sesión, registrar sus llaves de acceso a proveedores de inteligencia artificial (en la nube o locales) y elegir qué modelo usará cada agente. Asimismo, puede organizar sus conversaciones en proyectos, solicitar el diseño o análisis de un circuito en lenguaje natural y revisar los resultados en el visualizador de circuitos. Los diagramas de casos de uso detallados se encuentran en el apéndice A.

2.3. Arquitectura del sistema (modelo C4)

La arquitectura se describe mediante el modelo C4 en tres niveles: el contexto general, los contenedores que integran la plataforma y los componentes internos del servicio de agentes.

2.3.1. Nivel 1 — Contexto

El nivel de contexto ubica al sistema frente al usuario y los servicios externos con los que se comunica. El usuario ingresa desde su navegador para describir el circuito que desea y consultar el resultado validado. Por su parte, el sistema se conecta con el proveedor de inteligencia artificial configurado por el usuario para las tareas de lenguaje y con Langfuse para consultar las instrucciones de cada agente y registrar las trazas de ejecución, como se muestra en la figura 2.3.

Diagrama por agregar (SVG): `../src/diagrams/c4_l1_contexto.svg`

Elementos C4 Nivel 1: Usuario ↔ Sistema SPICE ↔ Sistemas externos (Proveedor de IA y Langfuse).

Figura 2.3. C4 Nivel 1: Diagrama de contexto del sistema.

2.3.2. Nivel 2 — Contenedores

El nivel de contenedores divide la plataforma en cuatro bloques principales que se levantan con Docker Compose: la aplicación web, el servidor de la aplicación, el servicio de agentes y la base de datos.

La **aplicación web** (desarrollada con React, Vite y TypeScript) ofrece el chat, la organización de proyectos y archivos, la elección del modelo de IA para cada agente y la visualización del esquema, las gráficas y el código SPICE. Se comunica con el servidor mediante HTTP/JSON para pedir y guardar datos, y recibe el avance de cada corrida en vivo por medio de eventos SSE (*Server-Sent Events*).

El **servidor de la aplicación** (construido con Bun, Hono, Drizzle y Better Auth) gestiona las cuentas y sesiones, los proyectos y conversaciones, las llaves de IA cifradas y el catálogo de circuitos. Se encarga de probar las llaves ante el proveedor de IA, iniciar cada corrida en el servicio de agentes (`POST /runs`) y reenviar su avance en vivo al navegador.

El **servicio de agentes** (implementado en Python con LangGraph, FastAPI y PySpice) coordina en un grafo a los seis agentes del sistema —Orquestador, Cálculo, Escritura, Simulación, Curador y Documentador— y ejecuta dentro del mismo contenedor el simulador `ngspice`. Durante una corrida, consulta al servidor el modelo asignado y el catálogo por una API interna, pide las instrucciones y registra la traza en Langfuse, consulta al proveedor de IA (Anthropic, OpenAI, Gemini, Ollama o LM Studio) y guarda los puntos de control del grafo.

Finalmente, la **base de datos** (PostgreSQL en Neon) almacena los usuarios, conversaciones, resultados, el catálogo de circuitos, las llaves cifradas y los puntos de control. La relación entre los cuatro contenedores y los sistemas externos se ilustra en la figura 2.4.

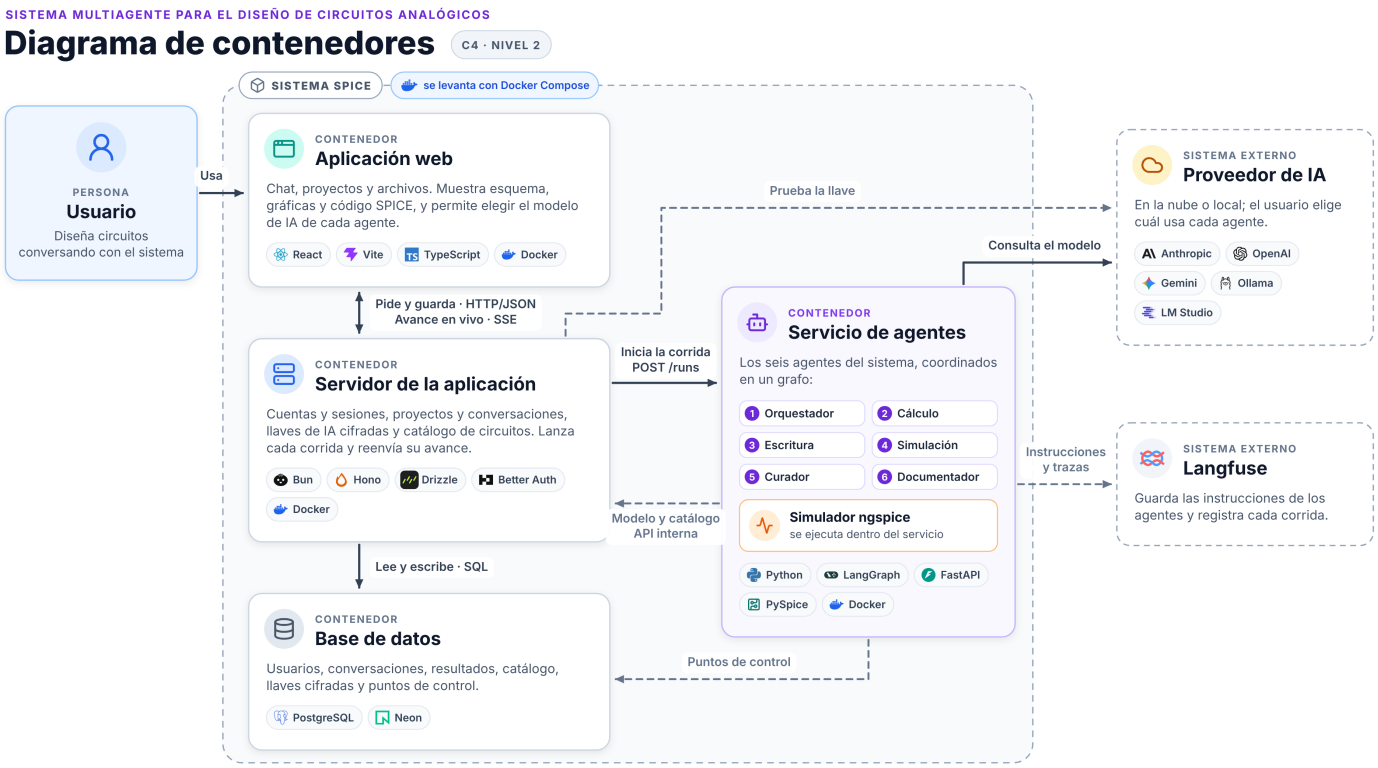


Figura 2.4. C4 Nivel 2: Diagrama de contenedores del sistema.

2.3.3. Nivel 3 — Componentes del servicio de agentes

Dentro del servicio de agentes, los componentes se agrupan en tres zonas: coordinación, construcción y validación. En la **zona de coordinación** se ubica el Orquestador, que recibe el pedido del usuario, consulta el catálogo de circuitos y define qué camino tomar (conversación, aclaración, análisis o diseño), acompañado por el nodo de Análisis —para resolver circuitos ya dados por el usuario— y el guardado de puntos de control.

En la **zona de construcción** se calculan los componentes, se ajustan a valores definidos y se redacta el archivo SPICE. Para el cálculo de circuitos de una o varias etapas se utiliza un esquema Maestro-Trabajador (*Master/Worker*) en paralelo: el Maestro de Cálculo recibe la especificación del Orquestador y crea un Trabajador independiente por cada sub-bloque del circuito. Cada Trabajador revisa su bloque: si pertenece al catálogo, calcula sus componentes con la fórmula correspondiente; y si es un bloque genérico, pasa tal cual el circuito propuesto por el modelo de lenguaje. Cuando todos los Trabajadores terminan, una barrera reúne los valores en el estado del circuito para avanzar a la etapa de Síntesis, como se muestra en la figura 2.5.

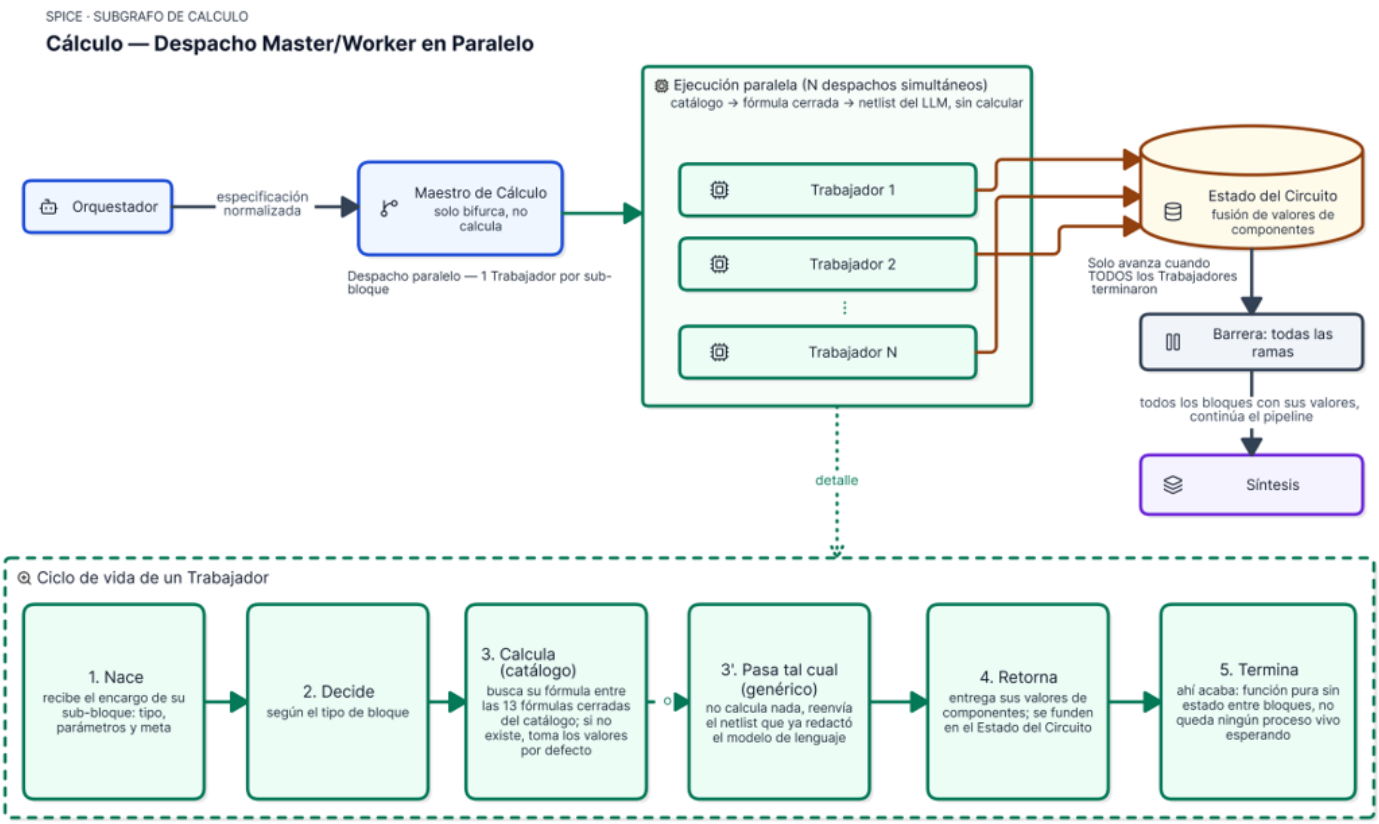


Figura 2.5. Subgrafo de Cálculo: despacho Maestro-Trabajador en paralelo y ciclo de vida de cada trabajador.

Una vez reunidos los valores ideales, se ajustan las resistencias, capacitores e inductores a valores definidos de series estándar junto con modelos definidos de semiconductores, y el agente de Escritura arma la lista de materiales y el archivo `.cir`.

Por último, en la **zona de validación y entrega**, el agente de Simulación corre el archivo en `ngspice` para leer las métricas y gráficas de respuesta, el Curador evalúa si se cumple la meta del usuario o si hace falta ajustar valores, y el Documentador genera la explicación final del diseño. La integración de estas tres zonas se presenta en la figura 2.6.

Diagrama por agregar (SVG): `../src/diagrams/c4_l3_agentes.svg`
Elementos C4 Nivel 3 (3 zonas): 1. Coordinación (Orquestador, Análisis y puntos de control); 2. Construcción (Maestro de Cálculo, Trabajadores en paralelo, ajuste a valores definidos y Escritura); 3. Validación y entrega (Simulación `ngspice`, Curador y Documentador).

Figura 2.6. C4 Nivel 3: Componentes del servicio de agentes.

2.4. Comportamiento dinámico: flujo de generación y validación

2.4.1. Máquina de estados y secuencia de comunicación

El control de flujo entre los agentes se rige por una máquina de estados finita que parte del estado inicial q_0 (Orquestador) y contempla tres caminos principales: diseño, análisis y conversación. Cuando el mensaje del usuario es una pregunta conceptual, requiere pedir datos faltantes o resulta inválido, el Orquestador responde y pasa directamente al estado final F. Cuando el usuario pide analizar un circuito existente, el flujo pasa al estado q_A (Análisis), donde se resuelve la red y se entrega el informe en F (o vuelve a q_0 si requiere corregir algún dato).

Si la solicitud es diseñar un circuito nuevo, el flujo avanza de q_0 a q_1 (Cálculo) para obtener los valores de cada bloque y enseguida a q_2 (Síntesis) para escribir el netlist y simularlo en `ngspice`. Con las métricas medidas se pasa a q_3 (Curador): si hace falta corregir el resultado, regresa a q_2 para ajustar y repetir la simulación; y una vez que emite su veredicto (aceptado o rechazado), avanza a q_4 (Documentador) para redactar la explicación final y terminar en F, tal como se ilustra en la figura 2.7.



Figura 2.7. Máquina de estados finita y función de transición para el control de flujo entre agentes.

En el tiempo, este recorrido se inicia cuando el usuario envía su solicitud desde la aplicación web al servidor, el cual delega la corrida al servicio de agentes y retransmite al navegador cada cambio de estado hasta entregar el circuito final, como se muestra en el diagrama de secuencia de la figura 2.8.

Diagrama por agregar (SVG): `../src/diagrams/secuencia_flujo_principal.svg`

Secuencia temporal: Usuario → Aplicación web → Servidor de la aplicación → Servicio de agentes (Orquestador → Cálculo → Lazo [Escritura → Simulación `ngspice` → Curador] → Documentador) → Servidor → Aplicación web.

Figura 2.8. Diagrama de secuencia del flujo principal de generación y validación.

2.4.2. Lazo de validación del agente Curador

El ciclo entre la simulación y el agente Curador asegura que el circuito cumpla en la práctica con lo solicitado. Al cambiar los valores calculados por valores definidos, la respuesta simulada en `ngspice` puede presentar una diferencia frente a la meta ideal. Por ello, en cada

intento n, el Curador compara el valor medido contra el esperado, obtiene el porcentaje de error (APE_i) y calcula un puntaje de evaluación:

$$R = -\sum_{i=1}^{M} w_i \cdot \text{APE}_i + \beta \cdot c - \gamma \cdot n \tag{2.1}$$

En esta expresión, el primer término descuenta el error porcentual de cada meta, el segundo término ($\beta \cdot c$) suma un bono si la simulación en `ngspice` convergió sin fallos, y el tercer término ($\gamma \cdot n$) resta puntos por cada intento adicional para favorecer soluciones rápidas dentro de una tolerancia del 5 %.

A partir de este resultado, el Curador toma una de tres decisiones: **aceptar** el diseño cuando el error está dentro de la tolerancia; **ajustar** el valor de los componentes (o combinar dos piezas en serie o paralelo si un solo valor redondeado no basta) para simular de nuevo; o **rechazar** la corrida si se alcanza el límite de intentos, entregando en ese caso la iteración que haya tenido el menor error. Este flujo de decisión se muestra en la figura 2.9.

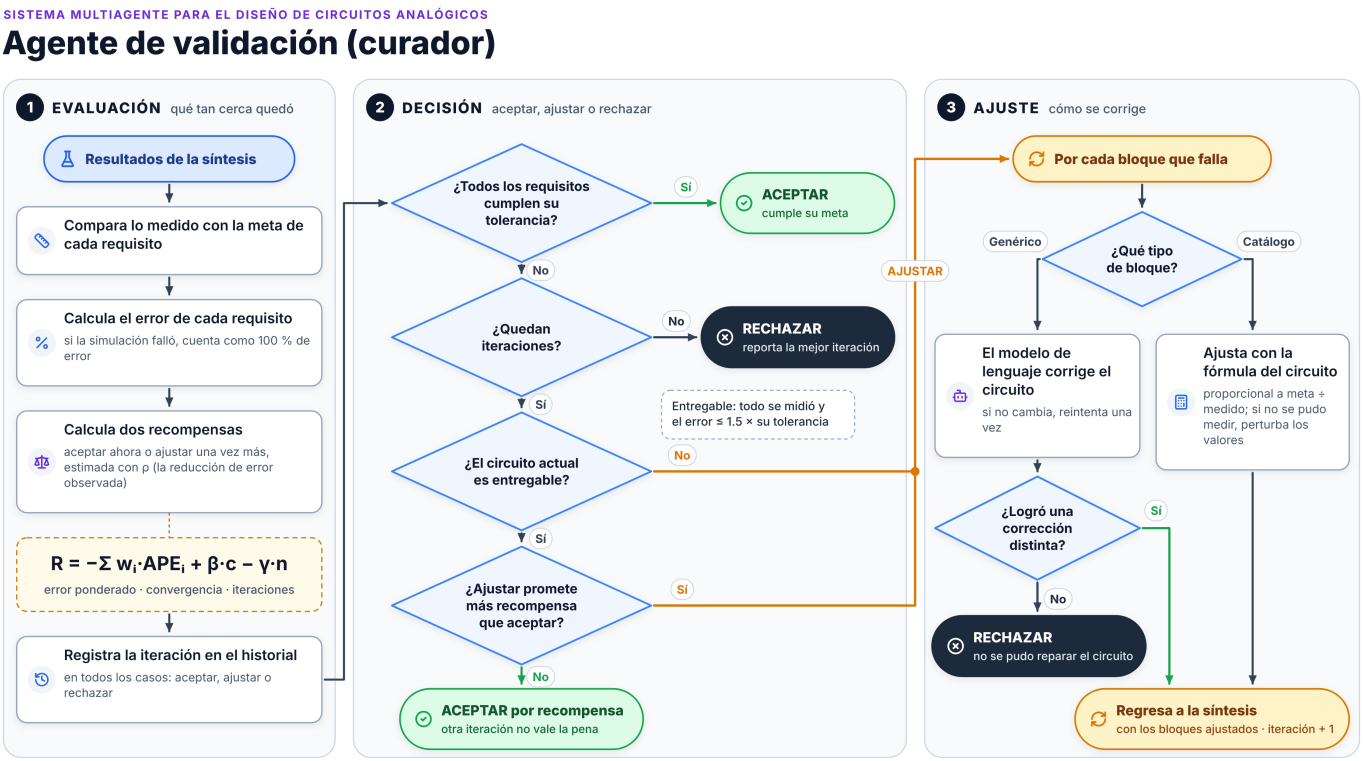


Figura 2.9. Flujo de evaluación, decisión y ajuste del agente Curador.

2.5. Desarrollo del sistema

Con base en el diseño anterior, se construyó el prototipo funcional del sistema, cuyo código fuente se encuentra publicado en el repositorio de GitHub https://github.com/antoniotorresxd/ia-spice. A continuación se describe el modelo de base de datos, el funcionamiento del catálogo de circuitos y de los valores definidos, y las pantallas que integran la aplicación web.

2.5.1. Modelo entidad-relación de la base de datos

La base de datos en PostgreSQL se diseñó para guardar las cuentas de usuario, la configuración de los modelos de inteligencia artificial y el historial de circuitos diseñados. La tabla central corresponde al usuario, la cual se relaciona con sus sesiones activas, sus cuentas de acceso y sus conexiones registradas hacia proveedores de IA; de estas conexiones dependen las asignaciones que indican qué modelo utiliza cada agente del sistema.

Para organizar el trabajo, cada usuario puede crear varios proyectos y agrupar dentro de ellos sus conversaciones. A su vez, cada conversación concentra tres tablas principales: los mensajes intercambiados en el chat entre el usuario y el asistente, el registro de cada corrida ejecutada (donde se guardan las mediciones de las simulaciones y las decisiones del Curador) y los archivos o artefactos finales del circuito (que almacenan el código SPICE, la lista de materiales y la explicación del diseño). Adicionalmente, se cuenta con una tabla para el catálogo de circuitos base del sistema y un espacio donde el grafo de agentes guarda sus puntos de control. La relación completa entre estas entidades se muestra en la figura 2.10.

Diagrama por agregar (SVG): `../src/diagrams/erd_base_datos.svg`

Diagrama Entidad-Relación (ERD): Usuario relacionado (1:N) con Sesiones, Cuentas, Conexiones de IA, Asignaciones de Modelo por Agente, Proyectos y Conversaciones; Conversación relacionada (1:N) con Mensajes, Ejecuciones y Artefactos; más la tabla de Catálogo de Circuitos.

Figura 2.10. Diagrama Entidad-Relación (ERD) de la base de datos en PostgreSQL.

2.5.2. Catálogo de circuitos, ajuste a valores definidos y modo de análisis

Para el diseño de circuitos, el sistema combina el uso de un catálogo base con el diseño de bloques abiertos. El catálogo reúne diecisiete circuitos analógicos clásicos divididos en cinco familias (divisores y fuentes, filtros pasivos y redes RLC, circuitos con diodos, etapas con transistores BJT y configuraciones con amplificadores operacionales), los cuales cuentan con sus ecuaciones de diseño listas para calcular los componentes de manera directa. Cuando

el usuario solicita un circuito que no forma parte del catálogo, el sistema lo trabaja como un bloque genérico en el que el modelo de lenguaje propone el circuito SPICE y lo envía al mismo ciclo de simulación en `ngspice` y revisión del Curador. Asimismo, cuando un diseño requiere conectar varias etapas en cascada —por ejemplo, un filtro seguido de un amplificador—, cada etapa se encapsula como un subcircuito y se enlaza la salida de una con la entrada de la siguiente dentro de un mismo archivo SPICE para simular el circuito completo. En la tabla 2.1 se muestran los diecisiete circuitos del catálogo.

Tabla 2.1. Catálogo de los 17 circuitos analógicos definidos en el sistema.

Categoría	**Circuito**	**Función principal y simulación SPICE**
Divisores y fuentes	Divisor resistivo de tensión	Reducción de voltaje en corriente directa (`.op`).
	Fuente regulada con zéner	Rectificación, filtrado y regulación de voltaje (`.tran`).
Filtros pasivos y RLC	Filtro pasabajos RC pasivo	Filtro de primer orden con frecuencia de corte (`.ac`).
	Filtro pasabanda RLC serie	Selección de una banda de frecuencia en serie (`.ac`).
	Circuito tanque RLC paralelo	Red resonante en paralelo (`.ac`).
Redes con diodos	Recortador simple con diodo	Recorte de voltaje a un nivel definido (`.tran`).
	Recortador de doble nivel	Recorte de voltaje en ambos semiciclos (`.tran`).
	Sujetador de nivel (*clamper*)	Desplazamiento del nivel de corriente directa (`.tran`).
Transistores BJT	Polarización por divisor	Ajuste del punto de operación en corriente directa (`.op`).
	Amplificador en emisor común	Amplificación de voltaje con transistor BJT (`.ac`/`.op`).
Amplificadores operacionales	Amplificador no inversor	Amplificación de voltaje manteniendo la fase.
	Amplificador inversor	Amplificación de voltaje invirtiendo la fase.
	Amplificador diferencial	Amplificación de la diferencia entre dos entradas.
	Amplificador de instrumentación	Amplificación diferencial con tres operacionales.
	Filtro pasaaltos activo	Filtro activo de primer orden con ganancia (`.ac`).
	Integrador con operacional	Integración de señal con red de estabilidad (`.ac`).
	Filtro Sallen-Key pasabajos	Filtro activo pasabajos de segundo orden (`.ac`).

Dado que las fórmulas de diseño entregan valores teóricos exactos que no siempre existen como piezas reales, antes de simular cada propuesta el sistema aproxima las resistencias, capacitores e inductores hacia los valores definidos más cercanos de las series estándar (E12, E24 y E96), o bien combina dos piezas en serie o en paralelo cuando se requiere mayor cercanía a la meta. Del mismo modo, asigna modelos SPICE definidos para los diodos, transistores

BJT y amplificadores operacionales, integrando todos los elementos en la lista de materiales del circuito.

Por otra parte, cuando el usuario pide analizar un circuito que ya tiene valores dados en lugar de diseñar uno nuevo, el módulo de análisis plantea y resuelve las ecuaciones del circuito (por nodos, mallas, reducciones serie y paralelo o equivalente de Thévenin y Norton) para obtener los voltajes, corrientes o ganancias buscadas, comprobando el resultado contra la simulación en `ngspice` antes de entregar la explicación paso a paso.

2.6. Pantallas del prototipo funcional

La aplicación web se divide en cinco vistas principales que guían al usuario desde el acceso a su cuenta hasta la consulta y descarga de los circuitos diseñados.

2.6.1. Pantalla de inicio de sesión y ruta de solución

La pantalla de inicio de sesión permite entrar al sistema o registrar una cuenta nueva en el panel derecho, con soporte para tema claro y oscuro. En su panel izquierdo muestra de forma visual la ruta de solución por la que pasa cada solicitud (desde el Orquestador y el Cálculo hasta la Síntesis y el Curador), como se observa en la figura 2.11.

Ruta de captura por agregar: ../src/pantallas/pantalla_login.png

Figura 2.11. Pantalla de inicio de sesión con el diagrama de la ruta de solución.

2.6.2. Configuración de proveedores de IA y modelos por agente

En la vista de modelos y proveedores, el usuario registra sus llaves de acceso para servicios en la nube o servidores locales, comprueba que la conexión funcione y elige qué modelo de inteligencia artificial utilizará cada agente del sistema (Orquestador, Cálculo, Escritura, Curador y Documentador), como se muestra en la figura 2.12.

Ruta de captura por agregar: ../src/pantallas/pantalla_modelos.png

Figura 2.12. Pantalla de configuración de proveedores de IA y asignación de modelos por agente.

2.6.3. Espacio de trabajo, nueva solicitud y avance en vivo

El espacio de trabajo principal reúne en una barra lateral los proyectos y las conversaciones del usuario, permitiendo organizarlas por carpeta y abrir un tutorial rápido sobre el uso de la interfaz, como se aprecia en la figura 2.13. Desde ahí se accede a la pantalla de nueva solicitud para escribir en lenguaje natural el circuito que se desea diseñar o partir de un ejemplo rápido, según se muestra en la figura 2.14.

Ruta de captura por agregar: ../src/pantallas/pantalla_workspace.png

Figura 2.13. Espacio de trabajo principal con proyectos, conversaciones y actividad reciente.

Ruta de captura por agregar: ../src/pantallas/pantalla_nueva_solicitud.png

Figura 2.14. Pantalla de creación de nueva solicitud en lenguaje natural.

Durante la ejecución de una corrida dentro de la conversación, un panel lateral muestra en vivo qué etapa está trabajando (interpretación, cálculo, simulación, curación o documentación), cuántos intentos de ajuste ha realizado el Curador y el acceso a la traza en Langfuse, como se presenta en la figura 2.15.

Ruta de captura por agregar: ../src/pantallas/pantalla_flujo_ejecucion.png

Figura 2.15. Panel de seguimiento en vivo de las etapas de ejecución y de los ajustes del Curador.

2.6.4. Archivos generados y Visualizador de circuitos

Los circuitos generados en las conversaciones pueden consultarse y filtrarse por proyecto en la vista de archivos y artefactos mostrada en la figura 2.16.

Ruta de captura por agregar: ../src/pantallas/pantalla_archivos.png

Figura 2.16. Vista del repositorio de archivos y circuitos SPICE generados.

Al seleccionar cualquiera de los circuitos se abre el visualizador del sistema. Esta vista integra en cinco pestañas toda la información del circuito validado: el diagrama esquemático dibujado automáticamente a partir del netlist; el código SPICE listo para copiarse o descargarse; la lista de materiales que compara el valor calculado con el valor definido; la gráfica interactiva con las curvas obtenidas de `ngspice`; y la explicación paso a paso de cómo se diseñó el circuito, como se ilustra en la figura 2.17.

Ruta de captura por agregar: ../src/pantallas/pantalla_visualizador.png

Figura 2.17. Visualizador de circuitos con diagrama esquemático, código SPICE, lista de materiales, gráfica de simulación y explicación del diseño.

Capítulo 3

Pruebas y Análisis de Resultados

Capítulo 4

Conclusiones y Trabajo Futuro

Referencias

[1] Z. Bao et al. «AnalogAgent: Self-Improving Analog Circuit Design Automation with LLM Agents.» arXiv: 2603.23910 [cs.AI]. dirección: https://arxiv.org/abs/2603.23910

[2] H. Zou, P. Han, E. Nazerian y A. Q. Huang. «PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards.» arXiv: 2602.00510 [cs.SE]. dirección: https://arxiv.org/abs/2602.00510

[3] K. S. Al Hasan, S. R. Raiyan, H. M. Alvee y W. Sadik. «CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts.» arXiv: 2601.04505 [cs.SE]. dirección: https://arxiv.org/abs/2601.04505

[4] Y. Lai et al., «AnalogCoder: Analog Circuit Design via Training-Free Code Generation,» en *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence (AAAI-25)*, 2025.

[5] Z. Wang, J. Gao, W. Fu, X. Guo y X. Zhang. «AnalogSAGE: Self-evolving Analog Design Multi-Agents with Stratified Memory and Grounded Experience.» arXiv: 2512.22435 [cs.AR]. dirección: https://arxiv.org/abs/2512.22435

[6] W. Qiu, Y. Xiao y R. Pan. «HWE-Bench: Can Language Models Perform Board-level Schematic Designs?» arXiv: 2603.18102 [cs.SE]. dirección: https://arxiv.org/abs/2603.18102

[7] Y. Yin, Y. Wang, B. Xu y P. Li. «ADO-LLM: Analog Design Bayesian Optimization with In-Context Learning of Large Language Models.» arXiv: 2406.18770 [cs.SE]. dirección: https://arxiv.org/abs/2406.18770

[8] D. Vungarala, S. Alam, A. Ghosh y S. Angizi. «SPICEPilot: Navigating SPICE Code Generation and Simulation with AI Guidance.» arXiv: 2410.20553 [cs.SE]. dirección: https://arxiv.org/abs/2410.20553

[9] Ngspice Development Team, *Ngspice User's Manual*, Version 44. dirección: https://ngspice.sourceforge.io/docs/ngspice-manual.pdf

[10] LangChain. «LangGraph: Building stateful, multi-actor applications with LLMs. »dirección: https://github.com/langchain-ai/langgraph

[11] F. Salvaire, *PySpice: Simulate electronic circuit using Python and the Ngspice/Xyce simulators*, 2021. dirección: https://github.com/PySpice-org/PySpice

[12] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] T. B. Brown et al., «Language Models Are Few-Shot Learners,» en *Advances in Neural Information Processing Systems (NeurIPS)*, verificar lista completa de autores, 2020.

[14] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, vol. 55, n.º 12, 2023, verificar volumen/número y páginas.

[15] M. Chen et al. «Evaluating Large Language Models Trained on Code.» verificar eprint y lista de autores. arXiv: 2107.03374 [cs.LG]. dirección: https://arxiv.org/abs/2107.03374

[16] R. S. Pressman, *Ingeniería del Software: Un Enfoque Práctico*, 7.ª ed. McGraw-Hill, 2010.

[17] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, 2000.

[18] A. S. Sedra y K. C. Smith, *Microelectronic Circuits*, 7.ª ed. Nueva York, EE. UU.: Oxford University Press, 2015, confirmar con Diego el libro de texto de referencia del dominio, ISBN: 978-0-19-933913-6.

[19] SkyWater Technology, *SkyWater SKY130 PDK*, 2023. dirección: https://skywater-pdk.readthedocs.io/en/main/index.html

[20] ISO/IEC, *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation*, International Organization for Standardization, 2011.

[21] W. Stallings y L. Brown, *Computer Security: Principles and Practice*, 4.ª ed. Pearson, 2018, opcional; ancla del principio CID. Alternativa: una publicación de NIST, ISBN: 978-0-13-479410-5.

[22] Better Auth. «Better Auth Documentation.» verificar URL y año contra la versión que uses. dirección: https://www.better-auth.com/

[23] OWASP Foundation. «OWASP Top 10: The Ten Most Critical Web Application Security Risks.» verificar año de la edición que cites; 2021 es la vigente. dirección: https://owasp.org/www-project-top-ten/

Apéndice A

Diagramas de casos de uso

Los diagramas complementarios del modelo de casos de uso presentado en la sección 2.2.2 se incorporarán en esta sección una vez actualizados conforme a los casos de uso implementados (acceso por sesión, configuración de modelos por agente, síntesis/análisis de circuitos y decisión del agente Curador).